

# HRMS – Leave Request

Complete Technical, Business Workflow, API & QA Documentation

**Document purpose:** Explain the Leave Request feature in simple but professional technical language so that an HR user, QA tester, frontend developer, backend developer, team lead, or new project member can understand the complete flow.

**Base URL:** <http://localhost:5168/>

**Implementation context:** ASP.NET Core / MediatR / Clean Architecture / Leave module / SQL Server. Company Holiday calculation is integrated into Create, Submit and Update leave flows.

**Important documentation rule:** This document records verified behavior from the current implementation and the test data already supplied during development. Where an exact endpoint contract or transition was not verified from the current source code, it is explicitly marked as “To be verified” instead of being invented.

## Document Contents

1. Purpose and Scope
2. Business Concepts
3. Actors
4. Leave Request Data Model
5. Statuses and State Machine
6. State Transition Rules
7. End-to-End Request Flow
8. Create Workflow
9. Update Workflow
10. Submit Workflow
11. Approve Workflow
12. Reject Workflow
13. Cancel Workflow
14. Delete Workflow
15. Leave-Day Calculation
16. Company Holiday Calculation
17. Validation and Business Rules
18. Error Handling
19. API Documentation Pattern
20. Database Verification
21. Sample End-to-End Scenario
22. QA Test Strategy
23. Positive Test Cases
24. Negative Test Cases
25. Regression Test Cases
26. Edge Cases
27. Developer Maintenance Guide
28. Production Readiness Checklist

## 1. Purpose and Scope

The Leave Request feature allows an employee to request leave for one or more dates, optionally using full-day or half-day day parts. The request begins as DRAFT, can be updated while editable, and can be submitted for approval. The system validates employee, leave year, leave type, day parts, dates, overlap and other business rules. Total leave days are calculated and company holidays are excluded when they are active, non-optional and not deleted for the selected Leave Year.

The scope includes the LeaveRequest entity, MediatR commands/handlers, LeaveRequestBusinessRules, repositories, status handling, day-part calculation and CompanyHoliday integration.

## 2. Business Concepts

**Leave Year:** The financial/leave period to which a request belongs. Example FY2027 is 2027-04-01 through 2028-03-31.

**Leave Type:** The configured category of leave used by the employee request.

**Day Part:** Defines how much of a day is requested. The current test data contains a full-day identifier and a half-day identifier.

**Company Holiday:** A date configured for a Leave Year that can reduce leave consumption when it is active, non-optional and not deleted.

**TotalDays:** The final leave consumption calculated by the application. It is not simply the number of calendar dates because day parts and eligible company holidays affect the result.

### 3. Actors

**Employee:** Creates a request, edits a DRAFT request, submits a request and performs other actions permitted by the current workflow.

**Approver/Manager:** Reviews submitted requests and performs approval/rejection where the current authorization and handler rules allow it.

**HR/Administrator:** Maintains leave configuration such as Leave Years and Company Holidays and can view/manage requests according to the application authorization model.

### 4. Current Test Data

**CompanyId:** 023FF1F8-4DD0-4942-1955-08DEFD16782B

**FY2026:** 3B320F3B-B8B4-4194-41CE-08DEFD305CC8 — 2026-04-01 to 2027-03-31

**FY2027:** 8BB8836A-3920-4DA2-41CF-08DEFD305CC8 — 2027-04-01 to 2028-03-31

**EmployeeId:** 19ED1DF1-C3CC-4614-85F0-08DEF7B598A7

**LeaveTypeId:** 9A5FF5C9-49DF-43F4-5107-08DEFD337F69

**Full-day DayPartId:** 30000000-0000-0000-0000-000000000001

**Half-day DayPartId:** 30000000-0000-0000-0000-000000000002

**Known company holiday examples:** 2027-04-23 and 2027-04-30 were used during testing.

### 5. Leave Request Data Model

| Field          | Meaning                              | Example          |
|----------------|--------------------------------------|------------------|
| Id             | Unique leave request identifier      | DAAA731C-...     |
| EmployeeId     | Employee who requests leave          | 19ED1DF1-...     |
| LeaveYearId    | Leave year                           | 8BB8836A-...     |
| LeaveTypeId    | Leave type                           | 9A5FF5C9-...     |
| StartDayPartId | Day part on first date               | 30000000-...001  |
| EndDayPartId   | Day part on final date               | 30000000-...001  |
| StatusId       | Current workflow status              | Status master ID |
| FromDate       | First leave date                     | 2027-04-21       |
| ToDate         | Last leave date                      | 2027-04-25       |
| TotalDays      | Calculated leave consumption         | 4.00             |
| Reason         | Employee reason                      | Personal work    |
| AppliedOn      | Application/submission timestamp     | ISO timestamp    |
| Audit fields   | Created/modified/deleted information | System-managed   |

## 6. Leave Request Status Model

The workflow is status-driven. A request must be in the correct status before an action is allowed. The exact status GUIDs are stored in the status master data; the workflow should use status meaning rather than hard-coded assumptions in clients.

Conceptual lifecycle:

```
CREATE → DRAFT → SUBMIT → PENDING → APPROVED
■ REJECTED
DRAFT/PENDING/other permitted state → CANCELLED
DRAFT → UPDATE → DRAFT
```

**DRAFT:** Request is being prepared and is the editable stage. The verified implementation explicitly rejects update when the request is not DRAFT.

**PENDING:** Request has been submitted and is waiting for approval/rejection. The verified implementation rejects update of a non-DRAFT request.

**APPROVED:** Final approval state. Exact post-approval actions should follow the current handler rules.

**REJECTED:** Request has been rejected. Exact re-submission behavior should follow the current implementation.

**CANCELLED:** Request has been cancelled. Exact allowed source states should be verified from the current Cancel handler.

## 7. State Transition Matrix

| Current   | Update                | Submit              | Approve             | Reject              | Cancel                 |
|-----------|-----------------------|---------------------|---------------------|---------------------|------------------------|
| DRAFT     | Allowed               | Allowed             | Not applicable      | Not applicable      | Verify current handler |
| PENDING   | Rejected              | Not applicable      | Allowed by workflow | Allowed by workflow | Verify current handler |
| APPROVED  | Rejected              | Not applicable      | Not applicable      | Not applicable      | Verify current handler |
| REJECTED  | Not normally editable | Not normally submit | Not applicable      | Not applicable      | Not applicable         |
| CANCELLED | Not allowed           | Not allowed         | Not allowed         | Not allowed         | Not allowed            |

## 8. End-to-End Technical Flow

```
Client ↓ API Controller ↓ MediatR Command ↓ Command Validator ↓ Command Handler ↓ LeaveRequestBusinessRules ■■
Employee validation ■■ Leave Year validation ■■ Leave Type validation ■■ Day-part validation ■■ Date validation ■■
Day-part combination validation ■■ Overlap validation ■■ Holiday-aware day calculation ■■ Status validation ↓
Repository ↓ EF Core / DbContext ↓ SQL Server
```

The important architectural rule is that the handler orchestrates the use case while `LeaveRequestBusinessRules` owns business validation/calculation logic. The repository handles persistence/query concerns.

## 9. CREATE Leave Request

**Purpose:** Create a new leave request in DRAFT state.

**Endpoint:** POST /api/leave-requests

**Verified handler sequence:**

1. Validate employee.
2. Validate Leave Year.
3. Validate Leave Type.
4. Validate start day part.
5. Validate end day part.
6. Validate dates against Leave Year.
7. Validate day-part combination.
8. Prevent overlapping PENDING/APPROVED leave.
9. Calculate TotalDays using holiday-aware calculation.
10. Obtain DRAFT status.
11. Create and persist LeaveRequest.

## Sample request:

```
{ "employeeId": "19ED1DF1-C3CC-4614-85F0-08DEF7B598A7", "leaveYearId": "8BB8836A-3920-4DA2-41CF-08DEFD305CC8",  
  "leaveTypeId": "9A5FF5C9-49DF-43F4-5107-08DEFD337F69", "startDayPartId": "30000000-0000-0000-0000-000000000001",  
  "endDayPartId": "30000000-0000-0000-0000-000000000001", "fromDate": "2027-04-21", "toDate": "2027-04-25", "reason":  
  "Personal work" }
```

If 2027-04-23 is an eligible company holiday, the expected TotalDays is 4.00 rather than 5.00.

The create response returns the new request identifier according to the current command contract. The exact response envelope should be taken from the current controller/exception middleware if a formal API response wrapper is used.

## 10. CREATE Database Result

Example conceptual database result: Id = EmployeeId = 19ED1DF1-C3CC-4614-85F0-08DEF7B598A7 LeaveYearId = 8BB8836A-3920-4DA2-41CF-08DEFD305CC8 FromDate = 2027-04-21 ToDate = 2027-04-25 TotalDays = 4.00 Status = DRAFT Reason = Personal work

## 11. UPDATE Leave Request

**Purpose:** Change an existing DRAFT request. The current implementation explicitly enforces the rule: only DRAFT leave requests can be updated.

**Endpoint pattern:** PUT/PATCH /api/leave-requests/{id} — use the exact verb currently exposed by the controller.

Verified negative example: attempting to update a non-DRAFT request returns 409 Conflict with:

```
{ "StatusCode": 409, "Message": "Only draft leave requests can be updated.", "Timestamp":  
  "2026-08-21T06:09:58.7315522+00:00", "TraceId": "0HNNV9ATR3F0V:00000002", "Errors": null }
```

Update recalculates TotalDays using the holiday-aware method. Therefore changing dates can change the number of consumed days.

Verified test example: request A6562C90-4F07-49D9-B1AF-08DEFED676EE was tested with 2027-04-28 → 2027-05-02 and an eligible holiday on 2027-04-30. Result: TotalDays=4.00.

## 12. SUBMIT Leave Request

**Purpose:** Submit a DRAFT request for approval. Submit revalidates and recalculates the request before changing its workflow status.

**Endpoint:** POST /api/leave-requests/{id}/submit

**Verified example:** POST /api/leave-requests/DAAA731C-86A6-4F2F-1EB4-08DEFF48E482/submit

**Verified sequence:** load request → validate current state/business rules → validate related data → recalculate TotalDays with LeaveYearId and CompanyHoliday rules → perform overlap checks → change status to submission state.

For the tested 2027-04-21 → 2027-04-25 request with one eligible company holiday, the calculated value is 4.00.

## 13. Why Submit Recalculates

A leave request may be created before HR adds or changes a company holiday. Therefore Create cannot be the only calculation point.

Example: Day 1: Employee creates 2027-05-10 → 2027-05-12. At creation there is no holiday. TotalDays = 3.00. Day 2: HR adds 2027-05-11 as a company holiday. Day 3: Employee submits. Submit recalculates: May 10 = 1 May 11 = 0 May 12 = 1 Final TotalDays = 2.00.

## 14. APPROVE

Conceptual workflow: PENDING → APPROVED.

The current project contains an ApproveLeaveRequestCommand/handler. The professional rule is to document its exact validation and response contract from the current source before treating any particular request JSON or response as authoritative. The state transition should only occur when the request is in the allowed source status.

Recommended technical sequence: load request → validate request exists → validate current status → execute approval business rules → change status → persist → return command result.

## 15. REJECT

Conceptual workflow: PENDING → REJECTED.

The current project contains a `RejectLeaveRequestCommand`/handler. The exact request DTO fields, rejection-reason requirement, endpoint verb and response envelope must follow the current controller/command contract. Do not allow a client to invent or bypass status rules.

## 16. CANCEL

Conceptual workflow: permitted active state → CANCELLED.

The current project contains a `CancelLeaveRequestCommand`/handler. The exact allowed source states and endpoint contract should be taken from the current implementation. Cancellation must be treated as a business transition, not merely a database status update.

## 17. DELETE

`LeaveRequest` has audit/soft-delete fields in the current database model. The Delete command should therefore be documented as a business operation that follows the module's existing soft-delete conventions, rather than assuming physical deletion.

The exact Delete endpoint and response should be taken from the current controller contract.

## 18. Leave-Day Calculation

The original synchronous calculation method remains available as the baseline. A holiday-aware asynchronous calculation was added so that the system can query `CompanyHoliday` records for the selected Leave Year.

**Current eligible company holiday filter:** `LeaveYearId` matches request AND `HolidayDate` is within `FromDate/ToDate` AND `IsDeleted=false` AND `IsActive=true` AND `IsOptional=false`.

The calculation starts from the existing day-part calculation and then removes eligible company-holiday days. A single-day leave that falls on a full company holiday is reduced to zero in the current holiday-aware method.

The implementation uses the start/end day-part value when a holiday falls on the first or last date, and one full day for a holiday strictly inside the range. Half-day-on-holiday behavior must be verified as an explicit business requirement.

## 19. Calculation Examples

### Example A — No holiday

2027-05-05 → 2027-05-07 = 3.00 days.

### Example B — One holiday in middle

2027-04-28 → 2027-05-02, holiday 2027-04-30:  $1+1+0+1+1 = 4.00$  days. This scenario was successfully tested.

### Example C — Single holiday day

Holiday 2027-05-10, leave 2027-05-10 → 2027-05-10: expected 0.00 for a full-day request.

### Example D — Multiple holidays

Leave 2027-05-10 → 2027-05-14; holidays May 10, 12, 14: expected 2.00 days.

## 20. Company Holiday Rules

A company holiday affects leave calculation only when all current filters are satisfied: same LeaveYear, date inside leave range, IsDeleted=false, IsActive=true, IsOptional=false.

Therefore an optional holiday, inactive holiday, deleted holiday, or holiday belonging to another LeaveYear must not automatically reduce leave consumption under the current calculation.

## 21. Validation and Business Rules

Employee validation: employee must exist and be valid for the request.

Leave Year validation: selected Leave Year must exist and dates must satisfy the Leave Year rules.

Leave Type validation: selected Leave Type must exist and be valid.

Day-part validation: start and end day parts must be valid.

Date validation: FromDate/ToDate must satisfy date ordering and Leave Year rules.

Day-part combination validation: start/end day-part combination must be valid for the selected date range.

Overlap validation: current implementation prevents an employee from creating/processing a request that overlaps a PENDING or APPROVED request.

Status validation: operations are allowed only in appropriate workflow states. Verified example: a PENDING request cannot be updated.

## 22. Verified Error Example — Overlap

An existing request for the same employee covering 2027-04-21 → 2027-04-25 was used to test an overlapping request. The application returned:

```
{ "StatusCode": 409, "Message": "Employee already has a pending or approved leave request for the selected dates.",  
  "Timestamp": "2026-08-21T06:10:58.5568162+00:00", "TraceId": "0HNNV9ATR3F0V:00000004", "Errors": null }
```

This is a successful business-rule rejection, not an application failure.

## 23. API Error Handling Standard

The project uses a structured error response containing StatusCode, Message, Timestamp, TraceId and Errors. QA should verify both the HTTP status and the business message.

Common categories: 400 validation error; 404 resource not found; 409 business-rule conflict; 500 unexpected server error. Exact status must follow the current exception middleware mapping.

## 24. Database Verification

After Create/Update/Submit, QA should verify the stored request.

```
SELECT Id, EmployeeId, LeaveYearId, LeaveTypeId, StartDayPartId, EndDayPartId, StatusId, FromDate, ToDate, TotalDays,  
Reason, AppliedOn FROM LeaveRequests WHERE Id = 'DAAA731C-86A6-4F2F-1EB4-08DEFF48E482';
```

For CompanyHoliday verification:

```
SELECT Id, LeaveYearId, HolidayDate, Name, HolidayType, IsOptional, IsActive, IsDeleted FROM CompanyHolidays WHERE  
LeaveYearId = '8BB8836A-3920-4DA2-41CF-08DEFD305CC8' ORDER BY HolidayDate;
```

## 25. Complete End-to-End Example

### Step 1 — Configure Leave Year

FY2027 = 2027-04-01 → 2028-03-31.

### Step 2 — Configure Company Holiday

2027-04-23 = Company Foundation/Company Holiday, active and non-optional.

### Step 3 — Employee creates leave

2027-04-21 → 2027-04-25.

**Step 4 — System validates**

Employee, Leave Year, Leave Type, day parts, dates, day-part combination and overlap.

**Step 5 — System calculates**

5 calendar days minus 1 eligible company holiday = 4.00.

**Step 6 — Request stored**

Status = DRAFT, TotalDays = 4.00.

**Step 7 — Employee submits**

System recalculates before submission.

**Step 8 — Request becomes pending**

Status = PENDING, TotalDays remains 4.00 if no relevant data changed.

**Step 9 — Approver reviews**

Approve or reject according to the workflow rules.

**Step 10 — Final database verification**

Verify status, TotalDays, audit information and dates in SQL Server.

## 26. QA Test Strategy

Testing should be performed in four layers: functional positive tests, negative/business-rule tests, calculation edge cases, and regression tests.

A test is PASS only when the API result, business behavior and database state agree with the expected result.

## 27. Positive Test Cases

| ID  | Scenario                                     | Expected                                        |
|-----|----------------------------------------------|-------------------------------------------------|
| P01 | Create normal leave                          | DRAFT, correct TotalDays                        |
| P02 | Create leave with one middle company holiday | Holiday excluded; 4.00 for proven 5-day example |
| P03 | Update DRAFT dates                           | Update succeeds and recalculates                |
| P04 | Submit DRAFT                                 | Submission succeeds and recalculates            |
| P05 | Full-day leave                               | 1.00 for one normal day                         |
| P06 | Half-day leave                               | 0.50 for one normal day                         |
| P07 | Multiple eligible holidays                   | All eligible holiday dates excluded             |
| P08 | Same holiday date in different LeaveYear     | Should be independent by LeaveYear              |
| P09 | Get paged leave list                         | Correct items and paging metadata               |
| P10 | Approve PENDING                              | Moves to APPROVED if business rules pass        |
| P11 | Reject PENDING                               | Moves to REJECTED if business rules pass        |
| P12 | Cancel permitted request                     | Moves to CANCELLED if current handler permits   |

## 28. Negative Test Cases

| ID  | Scenario               | Expected             |
|-----|------------------------|----------------------|
| N01 | Invalid EmployeeId     | Rejected; no request |
| N02 | Invalid LeaveYearId    | Rejected; no request |
| N03 | Invalid LeaveTypeId    | Rejected             |
| N04 | Invalid StartDayPartId | Rejected             |
| N05 | Invalid EndDayPartId   | Rejected             |
| N06 | FromDate after ToDate  | Rejected             |

| ID  | Scenario                                        | Expected                                      |
|-----|-------------------------------------------------|-----------------------------------------------|
| N07 | Dates outside LeaveYear                         | Rejected if current rule applies              |
| N08 | Invalid day-part combination                    | Rejected                                      |
| N09 | Overlap with PENDING leave                      | 409 conflict                                  |
| N10 | Overlap with APPROVED leave                     | 409 conflict                                  |
| N11 | Update PENDING request                          | 409: Only draft leave requests can be updated |
| N12 | Submit invalid/non-DRAFT request                | Rejected according to current status rule     |
| N13 | Unknown request ID                              | 404/not found                                 |
| N14 | Invalid GUID in request                         | 400 validation                                |
| N15 | Duplicate CompanyHoliday date in same LeaveYear | Rejected                                      |
| N16 | Optional holiday                                | Must not be deducted automatically            |
| N17 | Inactive holiday                                | Must not be deducted                          |
| N18 | Deleted holiday                                 | Must not be deducted                          |
| N19 | Holiday from another LeaveYear                  | Must not be deducted                          |

## 29. Calculation & Edge-Case Tests

| ID  | Scenario                                  | Expected                                          |
|-----|-------------------------------------------|---------------------------------------------------|
| E01 | No holiday, 3 calendar days               | 3.00                                              |
| E02 | One holiday in middle                     | Calendar days minus 1                             |
| E03 | Holiday on first date                     | Start-day-part value removed                      |
| E04 | Holiday on last date                      | End-day-part value removed                        |
| E05 | Single-day full-day holiday               | 0.00                                              |
| E06 | Multiple holidays                         | Each eligible full holiday removed                |
| E07 | Optional holiday                          | Not removed                                       |
| E08 | Inactive holiday                          | Not removed                                       |
| E09 | Deleted holiday                           | Not removed                                       |
| E10 | Holiday from wrong LeaveYear              | Not removed                                       |
| E11 | Half-day request on normal day            | 0.50                                              |
| E12 | Half-day request on full holiday          | Business requirement must be explicitly confirmed |
| E13 | Holiday added after Create, before Submit | Submit must recalculate                           |

## 30. Regression Test Suite

After modifying holiday calculation or LeaveRequest handlers, execute the existing LeaveRequest suite again: normal Create; half-day Create; Update DRAFT; Submit; Approve; Reject; Cancel; Delete/soft delete; Get list; pagination; employee validation; LeaveYear validation; LeaveType validation; day-part validation; date validation; overlap with PENDING; overlap with APPROVED.

The purpose is to prove that adding holiday calculation did not break unrelated LeaveRequest behavior.

## 31. Known Tested Project Scenarios

**Draft request:** 00CD6B6D-B76F-4C81-DBCB-08DEFDEBFE43 — 2027-04-20 → 2027-04-20 — TotalDays 1.00.

**Holiday baseline draft:** A6562C90-4F07-49D9-B1AF-08DEFED676EE — 2027-04-21 → 2027-04-25 — original 5.00; later update scenario produced 4.00 with holiday-aware calculation.



**Holiday test:** DAAA731C-86A6-4F2F-1EB4-08DEFF48E482 — 2027-04-21 → 2027-04-25 — calculated 4.00 with one eligible holiday.

**Successful update test:** A6562C90-4F07-49D9-B1AF-08DEFED676EE updated/tested around 2027-04-28 → 2027-05-02 with holiday-aware result 4.00.

**Verified business-rule errors:** non-DRAFT update returns 409; overlapping PENDING/APPROVED leave returns 409.

## 32. Developer Maintenance Guide

When changing leave calculation, do not modify only the Create handler. Review Create, Submit and Update because all three can calculate TotalDays.

When changing CompanyHoliday behavior, verify the holiday filter, LeaveYear relationship, optional flag, active flag, soft-delete behavior and date boundaries.

When changing status workflow, update the command handler/business rules and regression-test every permitted and forbidden transition.

When changing repository signatures, search every GetPagedAsync/GetAllAsync/FindAsync usage before compiling because the Leave module contains multiple query handlers.

Keep business rules in the business-rule layer rather than duplicating holiday/date logic in controllers.

## 33. Production Readiness Checklist

- All positive tests pass
- All negative tests pass
- First-day holiday tested
- Last-day holiday tested
- Single-day holiday tested
- Multiple holidays tested
- Optional/inactive/deleted holidays tested
- Wrong LeaveYear holiday tested
- Half-day tests completed
- Create/Submit/Update recalculation verified
- Overlap regression passes
- Status transition regression passes
- SQL data verified
- API response verified
- Logging/Traceld verified for errors
- No existing LeaveRequest functionality is broken

## 34. Final Architecture Summary

The Leave Request module follows a command-driven application flow: API → MediatR Command → Handler → Business Rules → Repository → EF Core → SQL Server. The handler coordinates the use case. BusinessRules performs domain/application validation and calculations. CompanyHoliday is queried through IReadRepository. TotalDays is recalculated at important workflow boundaries so that the current holiday calendar is respected.

The design keeps the existing LeaveRequest functionality intact while extending the calculation with CompanyHoliday awareness.

## 35. Documentation Status

**Current state:** LeaveRequest coding is complete and the major CompanyHoliday calculation scenarios have been tested. Full QA regression remains the final validation step before declaring the feature production-ready.

End of Technical Documentation